

# TensorFlow.js：使用遷移學習技術辨識音訊

來源：<https://codelabs.developers.google.com/codelabs/tensorflowjs-audio-codelab?hl=zh-tw>

步驟數：10

在本程式碼研究室中，您將建構可辨識您聲音的基本音訊辨識網路，並用來控制瀏覽器的滑桿。您將使用 TensorFlow.js，這個強大且彈性的 JavaScript 機器學習程式庫。

---

# 目錄

1. [1. 簡介](#)
2. [2. 需求條件](#)
3. [3. 載入 TensorFlow.js 和 Audio 模型](#)
4. [4. 即時預測](#)
5. [5. 測試預測](#)
6. [6. 收集資料](#)
7. [7. 測試資料收集](#)
8. [8. 訓練模型](#)
9. [9. 即時更新滑桿](#)
10. [10. 測試最終應用程式](#)

步驟 1 · 約 3 分鐘

## 1. 簡介

在本程式碼研究室中，您將建構音訊辨識網路，並使用該網路發出聲音，控制瀏覽器中的滑桿。您將使用 TensorFlow.js，這是功能強大且彈性十足的 JavaScript 機器學習程式庫。

首先，您會載入並執行[預先訓練模型](#)，該模型可辨識 20 個語音指令。接著使用麥克風建立及訓練簡單的類神經網路，辨識你的聲音並讓滑桿向左或向右移動。

本程式碼研究室**不會**介紹語音辨識模型背後的理論，如要瞭解詳情，請參閱[這篇教學課程](#)。

我們也建立了本程式碼研究室中機器學習術語的[詞彙表](#)。

## 課程內容

- 如何載入預先訓練的語音指令辨識模型
- 如何使用麥克風進行即時預測
- 如何使用瀏覽器麥克風訓練及使用自訂語音辨識模型

讓我們進入正題吧

---

## 2. 需求條件

如要完成本程式碼研究室，請務必符合以下條件：

1. 使用新版 [Chrome](#) 或其他新式瀏覽器。
2. 文字編輯器，可在本機電腦上執行，也可以透過 [Codepen](#) 或 [Glitch](#) 等服務在網路上執行。
3. 熟悉 HTML、CSS、JavaScript 和 [Chrome 開發人員工具](#) (或您偏好的瀏覽器開發人員工具)。
4. 對類神經網路有高階概念性瞭解。如需簡介或複習，建議觀看 [3blue1brown 的這部影片](#)，或是 [Ashi Krishnan 的這部 JavaScript 深度學習影片](#)。

**注意：**如果您是在 CodeLab 資訊站，建議使用 [glitch.com](#) 完成本程式碼研究室。我們已設定[新手範例專案](#)供您重混，載入 tensorflow.js。

步驟 3 · 約 3 分鐘

### 3. 載入 TensorFlow.js 和 Audio 模型

在編輯器中開啟 `index.html`，並新增下列內容：



```
<html>
  <head>
    <script src="https://cdn.jsdelivr.net/npm/@tensorflow/tfjs"></script>
    <script src="https://cdn.jsdelivr.net/npm/@tensorflow-models/speech-commands"></script>
  </head>
  <body>
    <div id="console"></div>
    <script src="index.js"></script>
  </body>
</html>
```

第一個 `<script>` 標記會匯入 TensorFlow.js 程式庫，第二個 `<script>` 則會匯入預先訓練的[語音指令模型](#)。 `<div id="console">` 標記會用於顯示模型的輸出內容。

---

步驟 4 · 約 5 分鐘

## 4. 即時預測

接著，在程式碼編輯器中開啟/建立 **index.js** 檔案，並加入下列程式碼：

```
let recognizer;

function predictWord() {
  // Array of words that the recognizer is trained to recognize.
  const words = recognizer.wordLabels();
  recognizer.listen(({scores}) => {
    // Turn scores into a list of (score,word) pairs.
    scores = Array.from(scores).map((s, i) => ({score: s, word: words[i]}));
    // Find the most probable word.
    scores.sort((s1, s2) => s2.score - s1.score);
    document.querySelector('#console').textContent = scores[0].word;
  }, {probabilityThreshold: 0.75});
}

async function app() {
  recognizer = speechCommands.create('BROWSER_FFT');
  await recognizer.ensureModelLoaded();
  predictWord();
}

app();
```

步驟 5 · 約 5 分鐘

## 5. 測試預測

確認裝置有麥克風。請注意，這項功能也適用於手機！如要執行網頁，請在瀏覽器中開啟 **index.html**。如果您是使用本機檔案，必須啟動網路伺服器並使用 `http://localhost:port/`，才能存取麥克風。

如要在通訊埠 8000 啟動簡易網路伺服器，請執行下列指令：

```
python -m SimpleHTTPServer
```

下載模型可能需要一段時間，請耐心等待。模型載入後，頁面頂端會顯示一個字。模型經過訓練，可辨識 0 到 9 的數字，以及「左」、「右」、「是」、「否」等其他指令。

說出其中一個字詞。是否能正確辨識你說的字？使用 `probabilityThreshold` 控制模型觸發的頻率，0.75 表示模型在 75% 以上的信心聽到特定字詞時會觸發。

如要進一步瞭解語音指令模型及其 API，請參閱 Github 上的 [README.md](#)。

---

## 6. 收集資料

為了增加趣味性，我們將使用短促的聲音來控制滑桿，而不是完整的字詞！

您將訓練模型辨識 3 種不同的指令：「向左」、「向右」和「噪音」，這些指令會讓滑桿向左或向右移動。辨識「噪音」（無須採取任何行動）對於語音偵測至關重要，因為我們希望滑桿只在發出正確聲音時做出反應，而不是在一般說話和移動時。

1. 首先，我們需要收集資料。在 `<div id="console">` 前方的 `<body>` 標記內新增下列內容，為應用程式新增簡單的 UI：

```
<button id="left" onmousedown="collect(0)" onmouseup="collect(null)">Left</button>
<button id="right" onmousedown="collect(1)" onmouseup="collect(null)">Right</button>
<button id="noise" onmousedown="collect(2)" onmouseup="collect(null)">Noise</button>
```

2. 將下列內容新增至 `index.js`：

```
// One frame is ~23ms of audio.
const NUM_FRAMES = 3;
let examples = [];

function collect(label) {
  if (recognizer.isListening()) {
    return recognizer.stopListening();
  }
  if (label == null) {
    return;
  }
  recognizer.listen(async ({spectrogram: {frameSize, data}}) => {
    let vals = normalize(data.subarray(-frameSize * NUM_FRAMES));
    examples.push({vals, label});
    document.querySelector('#console').textContent =
      `${examples.length} examples collected`;
  }, {
    overlapFactor: 0.999,
    includeSpectrogram: true,
    invokeCallbackOnNoiseAndUnknown: true
  });
}

function normalize(x) {
  const mean = -100;
  const std = 10;
  return x.map(x => (x - mean) / std);
}
```

3. 從 `app()` 移除 `predictWord()`：



```
async function app() {
  recognizer = speechCommands.create('BROWSER_FFT');
  await recognizer.ensureModelLoaded();
  // predictWord() no longer called.
}
```

## 詳細說明

這段程式碼乍看之下可能有點複雜，因此接下來我們會詳細說明。

我們在 UI 中新增了三個按鈕，分別標示為「Left」、「Right」和「Noise」，對應於我們希望模型辨識的三個指令。按下這些按鈕會呼叫我們新加入的 `collect()` 函式，為模型建立訓練範例。

`collect()` 會將 `label` 與 `recognizer.listen()` 的輸出內容建立關聯。由於 `includeSpectrogram` 為 `true`，`recognizer.listen()` 會提供 1 秒音訊的原始聲譜圖 (頻率資料)，並分成 43 個影格，因此每個影格的音訊長度約為 23 毫秒：

```
recognizer.listen(async ({spectrogram: {frameSize, data}}) => {
  ...
}, {includeSpectrogram: true});
```

由於我們想使用短促的聲音 (而非字詞) 控制滑桿，因此只會考量最後 3 個影格 (約 70 毫秒)：

```
let vals = normalize(data.subarray(-frameSize * NUM_FRAMES));
```

為避免發生數值問題，我們會將資料標準化，使平均值為 0，標準差為 1。在這種情況下，聲譜圖值通常是 -100 左右的負數，偏差值為 10：

```
const mean = -100;
const std = 10;
return x.map(x => (x - mean) / std);
```

最後，每個訓練範例都會有 2 個欄位：

- **label \*\*\*\***：分別以 0、1 和 2 表示「左」、「右」和「噪音」。
- **vals \*\*\*\***：696 個數字，包含頻率資訊 (頻譜圖)

並將所有資料儲存在 `examples` 變數中：

```
examples.push({vals, label});
```

---

## 7. 測試資料收集

**附註：**測試資料收集完畢後，您會丟棄樣本，因此請勿浪費時間收集過多樣本！

在瀏覽器中開啟 **index.html**，您應該會看到 3 個按鈕，分別對應 3 個指令。如果您是使用本機檔案，必須啟動網路伺服器並使用 `http://localhost:port/`，才能存取麥克風。

如要在通訊埠 8000 啟動簡易網路伺服器，請執行下列指令：

```
python -m SimpleHTTPServer
```

如要收集每個指令的範例，請**按住**每個按鈕 3 到 4 秒，同時重複 (或持續) 發出一致的聲音。每個標籤應收集約 150 個範例。舉例來說，我們可以以彈指聲代表「左」、口哨聲代表「右」，並以無聲和說話交替代表「噪音」。

收集更多範例後，頁面上顯示的計數器應該會增加。您也可以對 `examples` 變數呼叫 `console.log()`，檢查資料。此時的目標是測試資料收集程序。稍後測試整個應用程式時，您會重新收集資料。

---

步驟 8 · 約 10 分鐘

## 8. 訓練模型

1. 在 `index.html` 的主體中，於「**Noise**」按鈕後方新增「**Train**」按鈕：

```
<br/><br/>
<button id="train" onclick="train()">Train</button>
```

2. 在 `index.js` 的現有程式碼中加入下列內容：

```

const INPUT_SHAPE = [NUM_FRAMES, 232, 1];
let model;

async function train() {
  toggleButtons(false);
  const ys = tf.oneHot(examples.map(e => e.label), 3);
  const xsShape = [examples.length, ...INPUT_SHAPE];
  const xs = tf.tensor(flatten(examples.map(e => e.vals)), xsShape);

  await model.fit(xs, ys, {
    batchSize: 16,
    epochs: 10,
    callbacks: {
      onEpochEnd: (epoch, logs) => {
        document.querySelector('#console').textContent =
          `Accuracy: ${logs.acc * 100}.toFixed(1)}% Epoch: ${epoch + 1}`;
      }
    }
  });
  tf.dispose([xs, ys]);
  toggleButtons(true);
}

function buildModel() {
  model = tf.sequential();
  model.add(tf.layers.depthwiseConv2d({
    depthMultiplier: 8,
    kernelSize: [NUM_FRAMES, 3],
    activation: 'relu',
    inputShape: INPUT_SHAPE
  }));
  model.add(tf.layers.maxPooling2d({poolSize: [1, 2], strides: [2, 2]}));
  model.add(tf.layers.flatten());
  model.add(tf.layers.dense({units: 3, activation: 'softmax'}));
  const optimizer = tf.train.adam(0.01);
  model.compile({
    optimizer,
    loss: 'categoricalCrossentropy',
    metrics: ['accuracy']
  });
}

function toggleButtons(enable) {
  document.querySelectorAll('button').forEach(b => b.disabled = !enable);
}

function flatten(tensors) {
  const size = tensors[0].length;
  const result = new Float32Array(tensors.length * size);
  tensors.forEach((arr, i) => result.set(arr, i * size));
}

```

```
return result;
}
```

3. 應用程式載入時呼叫 `buildModel()`：

```
async function app() {
  recognizer = speechCommands.create('BROWSER_FFT');
  await recognizer.ensureModelLoaded();
  // Add this line.
  buildModel();
}
```

這時重新整理應用程式，就會看到新的「訓練」按鈕。您可以重新收集資料並點選「訓練」，藉此測試訓練作業，也可以等到步驟 10 再測試訓練和預測作業。

## 詳細說明

大致來說，我們要做兩件事：`buildModel()` 定義模型架構，以及 `train()` 使用收集到的資料訓練模型。

### 模型架構

這個模型有 4 個層：處理音訊資料 (以聲譜圖表示) 的卷積層、最大集區層、扁平層，以及對應至 3 個動作的稠密層：

```
model = tf.sequential();
model.add(tf.layers.depthwiseConv2d({
  depthMultiplier: 8,
  kernelSize: [NUM_FRAMES, 3],
  activation: 'relu',
  inputShape: INPUT_SHAPE
}));
model.add(tf.layers.maxPooling2d({poolSize: [1, 2], strides: [2, 2]}));
model.add(tf.layers.flatten());
model.add(tf.layers.dense({units: 3, activation: 'softmax'}));
```

模型的輸入形狀為 `[NUM_FRAMES, 232, 1]`，其中每個影格都是 23 毫秒的音訊，內含 232 個對應不同頻率的數字 (選擇 232 是因為這是擷取人聲所需的頻率值數量)。在本程式碼研究室中，我們使用 3 個影格的樣本 (約 70 毫秒的樣本)，因為我們是發出聲音，而不是說出完整單字來控制滑桿。

我們會編譯模型，準備進行訓練：

```
const optimizer = tf.train.adam(0.01);
model.compile({
  optimizer,
  loss: 'categoricalCrossentropy',
  metrics: ['accuracy']
});
```

我們使用 [Adam 最佳化器](#) (深度學習中常用的最佳化器)，並以 `categoricalCrossEntropy` 做為損失函式，這是用於分類的標準損失函式。簡單來說，這項指標會測量預測機率 (每個類別各有一個機率) 與真實類別機率為 100%，其他類別機率為 0% 的差距。我們也提供 `accuracy` 做為監控指標，可讓我們瞭解模型在每個訓練週期後，正確分類的樣本百分比。

## 訓練

訓練會使用大小為 16 的批次 (一次處理 16 個範例)，在資料上執行 10 次 (訓練週期)，並在 UI 中顯示目前的準確率：

```
await model.fit(xs, ys, {
  batchSize: 16,
  epochs: 10,
  callbacks: {
    onEpochEnd: (epoch, logs) => {
      document.querySelector('#console').textContent =
        `Accuracy: ${((logs.acc * 100).toFixed(1))}% Epoch: ${epoch + 1}`;
    }
  }
});
```

---

## 9. 即時更新滑桿

現在我們可以訓練模型了，請新增程式碼，即時做出預測並移動滑桿。在 **index.html** 中，於「Train」按鈕後方新增下列內容：

```
<br/><br/>
<button id="listen" onclick="listen()">Listen</button>
<input type="range" id="output" min="0" max="10" step="0.1">
```

以及 **index.js** 中的下列內容：

```
async function moveSlider(labelTensor) {
  const label = (await labelTensor.data())[0];
  document.getElementById('console').textContent = label;
  if (label == 2) {
    return;
  }
  let delta = 0.1;
  const prevValue = +document.getElementById('output').value;
  document.getElementById('output').value =
    prevValue + (label === 0 ? -delta : delta);
}

function listen() {
  if (recognizer.isListening()) {
    recognizer.stopListening();
    toggleButtons(true);
    document.getElementById('listen').textContent = 'Listen';
    return;
  }
  toggleButtons(false);
  document.getElementById('listen').textContent = 'Stop';
  document.getElementById('listen').disabled = false;

  recognizer.listen(async ({spectrogram: {frameSize, data}}) => {
    const vals = normalize(data.subarray(-frameSize * NUM_FRAMES));
    const input = tf.tensor(vals, [1, ...INPUT_SHAPE]);
    const probs = model.predict(input);
    const predLabel = probs.argmax(1);
    await moveSlider(predLabel);
    tf.dispose([input, probs, predLabel]);
  }, {
    overlapFactor: 0.999,
    includeSpectrogram: true,
    invokeCallbackOnNoiseAndUnknown: true
  });
}
```

# 詳細說明

## 即時預測

`listen()` 會聆聽麥克風的聲音，並即時預測。這段程式碼與 `collect()` 方法非常相似，可將原始聲譜圖標準化，並捨棄最後 `NUM_FRAMES` 個影格以外的所有影格。唯一的差別在於，我們也會呼叫訓練好的模型來取得預測結果：

```
const probs = model.predict(input);
const predLabel = probs.argmax(1);
await moveSlider(predLabel);
```

`model.predict(input)` 的輸出是形狀為 `[1, numClasses]` 的張量，代表類別數的機率分布。簡單來說，這就是一組可能輸出類別的可信度，總和為 1。張量的外部維度為 1，因為這是批次的大小 (單一範例)。

如要將機率分布轉換為代表最可能類別的單一整數，請呼叫 `probs.argmax(1)`，這會傳回機率最高的類別索引。我們將「1」做為軸參數傳遞，因為我們想在最後一個維度 `numClasses` 中計算 `argMax`。

## 更新滑桿

如果標籤為 0 (「左」)，`moveSlider()` 會降低滑桿的值；如果標籤為 1 (「右」)，則會提高滑桿的值；如果標籤為 2 (「噪音」)，則會忽略滑桿的值。

## 處置張量

如要清理 GPU 記憶體，請務必手動對輸出張量呼叫 `tf.dispose()`。手動 `tf.dispose()` 的替代做法是將函式呼叫包裝在 `tf.tidy()` 中，但這無法與 `async` 函式搭配使用。

```
tf.dispose([input, probs, predLabel]);
```

---



## 10. 測試最終應用程式

在瀏覽器中開啟 **index.html**，然後使用對應 3 個指令的 3 個按鈕，收集資料 (與上一節的做法相同)。請記得在收集資料時**按住**每個按鈕 3 到 4 秒。

收集完範例後，請按下「訓練」按鈕。這會開始訓練模型，您應該會看到模型的準確率超過 90%。如果模型成效不佳，請嘗試收集更多資料。

訓練完成後，按下「聆聽」按鈕，即可透過麥克風進行預測，並控制滑桿！

如需更多教學課程，請前往 <http://js.tensorflow.org/>。

---